

Continuing research and analysis

I need to continue the research from our previous discussion. The user is asking for updates, so I should likely analyze the problem using available tools, possibly doing some experimenting. Since they asked about continuing research, I won't focus solely on proofs. However, I may reference the previous context for clarity.

Setting up research progress and analysis

I need to continue exploring the research, possibly using Python. The goal is to provide progress on solving F56, making sure I keep the user updated regularly, every 15 seconds or after 2-3 tool calls. No need for clarification right now, but I might start with a rhetorical question instead. Let's focus on the task!

Formalizing the problem structure

I need to formalize the problem with vertices and edges representing different constraints. There are incident edge labels based on half-edge positions, giving each vertex a degree of 56. We might allow loops and multiple edges, but the algorithm should handle everything generically. I'm thinking of proving the majority-minority closure more rigorously, possibly using algebraic structures like 2-decomposable operations.

Exploring relational structures and algorithms

I'm considering finite relational structures and their properties, especially when relations are preserved by majority and minority operations. It seems these properties might give rise to binary constraints and potentially lead to polynomial algorithms, particularly through consistency checking. If the theory holds, this could simplify counting problems to counting components.

Exploring path consistency and EO instances

If an instance is (2,3)-minimal and path consistent, then partial assignments can extend to a solution, which implies closure for binary projections in majority-preserved relations. The problem becomes more complex given the subclass nature of the EO instances, necessitating a careful approach for applying strict width and compositional rules.

Clarifying majority operation on D

I'm considering whether the base edge relations are preserved by a majority operation on the domain D. Previously, we defined majority in terms of solution triples, but now it's tricky because, on arbitrary tuples, majority is undefined if all values are distinct. The theorem only guarantees that for solutions of a given instance, the triple of values won't be distinct across coordinates.

Exploring how majority and minority operations preserve relations

I'm thinking about whether majority and minority operations on Boolean elements preserve cut relations. For an operation m on D induced by a map M , the goal would be for M to preserve these relations if certain conditions hold. The XOR operation seems related, but there are challenges with closure under XOR of arbitrary elements. Maybe special operations could help preserve these cuts? I'll continue exploring Boolean linear operations projected onto G .

Exploring ternary majority operations and cut relations

I'm wondering if a fixed ternary operation could work on D using majority when two inputs are equal, and outputting the unique remaining value from a fixed structure when the inputs are distinct. Could this preserve cut relations like the NAE (Not-All-Equal) conditions and complete graph edges? I'll need to check if a conservative majority operation preserves these relations, especially those defined on structures like $S \times \bar{T}$ and $\bar{S} \times T$.

Exploring majority operation for preserving relations

I'm wondering if there's an operation that preserves relations when the block membership on each side aligns with the majority of memberships. The idea is that for a given subset A , the operation should match the majority membership. Could defining a fixed majority operation on D using 4-bit embeddings work? I'll need to investigate whether this works for every even subset of D , as they correspond to linear functionals in F_2^4 .

Testing majority operation with linear combinations

I'm trying to figure out if a coordinate-wise majority operation, defined as $t_j = \text{Maj}(u_j, v_j, w_j)$, satisfies the same properties for all linear functionals. Over $GF(2)$, the majority is represented by the polynomial $xy + xz + yz$, but does it commute with the functionals? Let's verify this with a test.

Testing coordinatewise majority for linear functionals

Code:

```
import itertools, numpy as np, math, collections
G = {
    1:(1,0,0,0),
    2:(0,1,0,0),
    3:(0,0,1,0),
    4:(0,0,0,1),
    5:(1,1,1,1),
}
vec_to_label = {v:k for k,v in G.items()}
def maj_bits(bits):
    return 1 if sum(bits)>=2 else 0
def xor_bits(bits):
    return sum(bits)%2
# test coordinatewise majority of any triple of G stays in G?
maj_count=0
bad=[]
for xs in itertools.product(G.values(), repeat=3):
```

```

t=tuple(maj_bits([x[i] for x in xs]) for i in range(4))
if t not in vec_to_label:
    bad.append((xs,t))
len(bad), bad[:10]

```

Output:

```

(60,
 [((1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0)), (0, 0, 0, 0)),
 ((1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 0, 1)), (0, 0, 0, 0)),
 ((1, 0, 0, 0), (0, 1, 0, 0), (1, 1, 1, 1)), (1, 1, 0, 0)),
 ((1, 0, 0, 0), (0, 0, 1, 0), (0, 1, 0, 0)), (0, 0, 0, 0)),
 ((1, 0, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1)), (0, 0, 0, 0)),
 ((1, 0, 0, 0), (0, 0, 1, 0), (1, 1, 1, 1)), (1, 0, 1, 0)),
 ((1, 0, 0, 0), (0, 0, 0, 1), (0, 1, 0, 0)), (0, 0, 0, 0)),
 ((1, 0, 0, 0), (0, 0, 0, 1), (0, 0, 1, 0)), (0, 0, 0, 0)),
 ((1, 0, 0, 0), (0, 0, 0, 1), (1, 1, 1, 1)), (1, 0, 0, 1)),
 ((1, 0, 0, 0), (1, 1, 1, 1), (0, 1, 0, 0)), (1, 1, 0, 0))])

```

Searching for alternative operations on D

I'm considering whether there's another operation on D that might satisfy the linear functional majority property. Since the coordinate-wise majority doesn't stay in G, and for triples with all distinct values the output is not determined by the coordinates, I need to explore. With a domain of size 5, the number of ternary operations is huge, but maybe constraints on even-subset membership can help. I'll try testing the existence of such operations by calculating the set of labels that satisfy all the equations for each triple.

Testing even-subset membership for ternary operations

Code:

```

# even subsets of D labels 1..5
labels=[1,2,3,4,5]
even_subsets=[]
for r in [0,2,4]:
    for comb in itertools.combinations(labels,r):
        even_subsets.append(set(comb))
len(even_subsets)

```

Output:

```
16
```

Testing ternary operation with majority constraint

Code:

```

def membership_majority_output(triple):

```

```

target={}
for S in even_subsets:
    bits=[1 if x in S else 0 for x in triple]
    target[frozenset(S)] = 1 if sum(bits)>=2 else 0
cands=[]
for t in labels:
    ok=True
    for S in even_subsets:
        if (1 if t in S else 0) != target[frozenset(S)]:
            ok=False; break
    if ok: cands.append(t)
return cands,target

bad=[]
for triple in itertools.product(labels, repeat=3):
    cands,_=membership_majority_output(triple)
    if len(cands)!=1:
        bad.append((triple,cands))
len(bad), bad[:20]

```

Output:

```

(60,
 [((1, 2, 3), []),
 ((1, 2, 4), []),
 ((1, 2, 5), []),
 ((1, 3, 2), []),
 ((1, 3, 4), []),
 ((1, 3, 5), []),
 ((1, 4, 2), []),
 ((1, 4, 3), []),
 ((1, 4, 5), []),
 ((1, 5, 2), []),
 ((1, 5, 3), []),
 ((1, 5, 4), []),
 ((2, 1, 3), []),
 ((2, 1, 4), []),
 ((2, 1, 5), []),
 ((2, 3, 1), []),
 ((2, 3, 4), []),
 ((2, 3, 5), []),
 ((2, 4, 1), []),
 ((2, 4, 3), [])])

```

Exploring the existence of a majority polymorphism

It looks like there's no fixed majority operation on all D that satisfies all even subsets—unsurprisingly. But maybe there's a majority polymorphism for a restricted set of relations. I'm considering a brute-force approach to count conservative majority operations on the 5 elements, looking for those that preserve certain relations. While searching 5^{60} is daunting, the constraints might make this simpler when

approached with CP-SAT. Alternatively, I could look at the structure of the raw relations.

Exploring ternary operations and their relation preservation

The relations are preserved by ternary operations on D that are polymorphisms of unary even-subset partitions. For each even subset S , the induced Boolean operation on membership bits ensures preservation of behavior. While a majority polymorphism may not exist, exploring the class of all EO instances might help show decomposability, though exactness isn't guaranteed across all binary networks. Let's see if we can prove exactness using the block-flip structure.

Investigating vertex-specific differences and constraints

It looks like I can possibly recover block directions from the linear solution space $A = x_0 + K$ intersected with G^V . Maybe K has a special form tied to the vertex-specific occurrence multiset. By considering the difference space of two solutions at each vertex, I'll check if differences give us a linear code over assignments. The set of possible nonzero differences at any vertex should be the 10 weight-2 or weight-4 secants. Let's explore this direction!

Computing vertex difference space and secants

Code:

```
Deltas={}
for i,j in itertools.combinations(labels,2):
    d=tuple((G[i][k]^G[j][k]) for k in range(4))
    Deltas[(i,j)] = d
collections.Counter(Deltas.values()), len(set(Deltas.values()))
```

Output:

```
(Counter({(1, 1, 0, 0): 1,
          (1, 0, 1, 0): 1,
          (1, 0, 0, 1): 1,
          (0, 1, 1, 1): 1,
          (0, 1, 1, 0): 1,
          (0, 1, 0, 1): 1,
          (1, 0, 1, 1): 1,
          (0, 0, 1, 1): 1,
          (1, 1, 0, 1): 1,
          (1, 1, 1, 0): 1}),
10)
```

Verifying difference solutions and cut-crossing patterns

It looks like each nonzero difference vector corresponds to a cut-crossing pattern in the graph. For vectors $d = g_i + g_j$, when we dot product with a_S , we get an indicator based on whether the edge ij crosses the cut S . Now, the condition $\text{cross}_S(\delta_u) = \text{cross}_T(\delta_v)$ should hold for edges between port types. This forms a 10-state constraint system. Let's check this further.

Exploring differences, graph representations, and matroids

If the nonzero difference at a vertex represents incidence of a K5 edge with a cut, then the multiset of ports could indicate shared cuts. Each edge enforces equality of one bit between K5 edges at endpoints. I'm curious if this could form a matroid over $\text{GF}(2)$, which might simplify the structure. The global difference delta should span a block of vertices where each has $\delta_v \neq 0$. I'll investigate further to see if this leads to a combinatorial description.

Exploring solutions and difference spaces

I'm wondering if the solutions to this problem could be simpler if we treat blocks as nonzero difference solutions. To investigate, I can compute the space of all pairwise differences in this CSP. Nonzero delta solutions might be unique up to picking one of 10 directions. I could try coding small instances in Python to explore this. Another idea is to use Fourier or holographic transformations, since the vertex signatures might be tractable by decomposing or changing the basis with matchgates.

Exploring partition functions and models

I'm looking into whether the exact partition function reduces to something like the Tutte polynomial or Ising model on a graph based on incidence pairings. The local dual signature might translate to the Potts model. I think I could represent the partition function as a sum over edge variables, with 56-tuples equal to certain rows. Using Fourier transforms here might help. Each edge could enforce disequality (XOR 1) between its endpoints, so let's compute it directly to fully understand this.

Examining partition function and edge constraints

I'm considering how the partition function might work. There's a possible form with terms involving the product over vertices and edges, incorporating Fourier transforms. I need to handle the orientation of half-edges and how edge disequality relates to the Walsh transform, specifically how it would become parity weight. Let's derive this with the constraint for each edge, considering the XOR condition between the endpoints.

Computing edge disequality Fourier transform

Code:

```
import numpy as np, itertools
M=np.array([[0,1],[1,0]]) # disequality
H=np.array([[1,1],[1,-1]])
print(H@M@H)
```

Output:

```
[[ 2  0]
 [ 0 -2]]
```

Considering dual model and partition function

I'm looking at how the transformation shifts this into a simpler diagonal model. After applying the transformation, each edge just has one bit associated, and the partition function takes a sum over these variables. It seems the result might become a binary edge-spin model with 4-bit syndrome at each vertex and local weights. I'm wondering if this could be tractably solved by contraction-deletion techniques.

Analyzing weight formula and consistency

I'm figuring out the weight function and testing different forms for consistency with the quadratic form. It's tricky because some of my initial assumptions don't align with expectations. I'm now considering using orthogonal geometry or exploring linear systems to better model the differences. It might be possible to derive an algorithm using blocks or intersection-theorem ideas. I'll need to verify the relationship between solution supports.

Exploring polynomial time options for decision problem

I'm wondering if I could compute this by solving multiple linear systems or using an oracle to check feasibility. The idea of generalized arc consistency keeps surfacing, which might be a more direct route. Still, testing the true feasibility needs a satisfiability algorithm, so I'll need to explore if 2-SAT or XOR-SAT might offer a solution.

Examining graph relations and homomorphisms

I'm exploring if I can represent the instance as a signed graph using cuts, where satisfiability and counting correspond to homomorphisms to K_5 with specific constraints. Each label could be a vertex, and edges would represent conditions about the cut membership. Could these constraints lead to equivalency relations over finite Abelian groups? I'm not sure yet!

Exploring graph homomorphisms and automorphisms

I'm wondering if there's a hidden connection between the tension/flow in K_5 and this solution. Could a solution be viewed as a graph homomorphism from a derived graph to K_5 ? The constraints depend on permutations of cuts and could be normalized using group actions. If I explore these automorphisms, I may simplify things!

Exploring equivalence to 5-coloring using K_5 cuts

I'm considering that the 5 row labels might correspond to 5 colors, with each port type representing incidence in K_5 . If each f56 vertex works like a "color choice" gadget, edges compare selected colors. There seems to be a connection to 5-coloring of a derived graph, considering cut constraints and transformations.

Exploring port multiset and support matrix properties

I'm wondering if the port multiset includes four copies of each pair cut and three copies of each complement-of-singleton cut, plus one all-zero column. The patterns on the rows might form a linear code. I'll check for orthogonality in the support matrix and consider row space and Gram matrix properties.

Computing support matrix for combinatorial design analysis

Code:

```
# construct columns as even subsets with multiplicities
cols=[]
for S in [frozenset()]:
    cols.append((S,1))
for S in itertools.combinations(labels,2):
    cols.extend([(frozenset(S),4)])
for j in labels:
    S=frozenset(set(labels)-{j})
    cols.extend([(S,3)])
len(cols)
```

Output:

```
16
```

Expanding tuples with multiplicity counts

Code:

```
expanded=[]
expanded += [frozenset()] * 1
for S in itertools.combinations(labels,2):
    expanded += [frozenset(S)] * 4
for j in labels:
    expanded += [frozenset(set(labels)-{j})] * 3
len(expanded)
```

Output:

```
56
```

Computing Hamming distances with expanded labels

Code:


```

rows={}
for i in labels:
    rows[i]=[1 if i in S else 0 for S in expanded]
# hamming distances
d={}
for i,j in itertools.combinations(labels,2):
    d[(i,j)] = sum(a^b for a,b in zip(rows[i],rows[j]))
d

```

Output:

```

{(1, 2): 30,
 (1, 3): 30,
 (1, 4): 30,
 (1, 5): 30,
 (2, 3): 30,
 (2, 4): 30,
 (2, 5): 30,
 (3, 4): 30,
 (3, 5): 30,
 (4, 5): 30}

```

Analyzing row distances and weights

I'm noticing that all rows are equidistant, at a distance of 30. I wonder about the weight of each row—how many 1s there are in each of them. I'll consider the structure of the rows and how their weight relates to the distances between them.

Calculating row weights

Code:

```

weights={i:sum(rows[i]) for i in labels}
weights

```

Output:

```

{1: 28, 2: 28, 3: 28, 4: 28, 5: 28}

```

Analyzing design and overlap properties

I'm working with a 2-(5,?) design where each row represents a 5-length codeword of length 56 and weight 28. The pairwise overlap seems to be 13, given the distance of 30 and weight of 28. This could be a regular simplex, so now I'm thinking of converting these to ± 1 vectors and checking for constant pairwise dot products.

Computing pairwise dot products and overlaps

Code:

```
pm={}
for i in labels:
    pm[i]=[1 if bit==0 else -1 for bit in rows[i]] # maybe +/-1
dots={}
for i,j in itertools.combinations(labels,2):
    dots[(i,j)] = sum(a*b for a,b in zip(pm[i],pm[j]))
sum(pm[1][k]*pm[1][k] for k in range(56)), dots[(1,2)]
```

Output:

```
(56, -4)
```

Exploring Potts model connections and vector interactions

The 5 row ± 1 vectors form a regular simplex in $\mathbb{R}^{56}$ with norm $\sqrt{56}$ and pairwise dot product -4. After centering, they may be orthogonal. This setup resembles a 5-state Potts model, where the edge weights depend on membership to certain cuts and can be viewed as interactions based on vector dot products.

Exploring the algebraic geometry connection

The 5 points might correspond to a 2-(5,2,1) design. The action of group elements on these points could relate to the permutation representation of S_5 and connections to A_5 . The model could represent spin representations of a 4D orthogonal group. This setup deeply ties into projective geometry $PG(3, 2)$ and orthogonal polar space $Q^-(3, 2)$.

Exploring geometric structures and affine subspaces

Looking at the

Investigating affine subspaces and intersections

I'm exploring whether local projections can have more than two points when intersecting with a nonzero singular vector space. These projections are intersected with affine subspaces in F_2^4 , and I'm wondering if this setup—possibly linked to elliptic quadric geometry—restricts the intersection to only two points. A brute force check might help clarify the situation.

Brute forcing affine subspace intersections in F_2^4

Code:

```
# enumerate all affine subspaces x+U of F2^4, intersect with G
allvec=[tuple(int(b) for b in f"{n:04b}") for n in range(16)]
def add(u,v): return tuple(a^b for a,b in zip(u,v))
def span(vs):
    s={ (0,0,0,0) }
```

```

    for v in vs:
        s |= {add(x,v) for x in list(s)}
    return s
# all subspaces
subs=set()
subspaces=[]
for r in range(5):
    for basis in itertools.combinations(allvec[1:], r):
        U=frozenset(span(basis))
        if len(U)==2**r and U not in subs:
            subs.add(U)
            subspaces.append(set(U))
len(subspaces)

```

Output:

```
67
```

Brute forcing affine subspace intersections with G.

Code:

```

Gset=set(G.values())
ints=collections.Counter()
examples={}
for U in subspaces:
    for x in allvec:
        A={add(x,u) for u in U}
        inter=A & Gset
        ints[len(inter)] += 1
        examples.setdefault(len(inter),(x,U,inter))
ints, examples.keys()

```

Output:

```

(Counter({1: 415, 2: 260, 0: 221, 3: 120, 4: 40, 5: 16}),
 dict_keys([0, 1, 2, 3, 4, 5]))

```

Refining conjectures on affine subspaces

I initially thought affine subspaces could only intersect in two points, but actually, they can intersect in more—three, four, or five. So, my conjecture about local projections $\dim \pi_v(K) \leq 1$ becomes nontrivial. The constraints from the EO conditions might introduce specific properties. We may need to examine how edge incidences limit projections at each vertex to derive the necessary conditions.

Exploring the difference space and connections

This problem seems like a constraint satisfaction issue about edge bits with vertex constraints. Each comes from row differences encoded as 56-bit patterns, where 11 patterns are possible based on XOR

interactions. The difference space might align with an even delta-matroid or linear matroid structure, which could help simplify counting solutions. Let's investigate further.

Refining the edge-bit solution process

So, each vertex having one of the 11 patterns makes things simpler, as the edge constraints are equalities, not disequalities. Subtracting two solutions converts edge disequality into equality. This gives a Holant problem where difference assignments match patterns, leading to easier counting. Let's formalize this system.

Thinking about global difference configurations and counting patterns

The global difference configurations involve assigning bits to each edge, ensuring each vertex's 56 incident bits match certain patterns. These patterns could form a 0-1 valued signature. It might be that these patterns align with a linear code or even a delta-matroid. Let's delve into whether H matches the cut-incidence vectors of K_5 .

Thinking through graph edge constraints and block-flip

Each graph edge enforces an equality between the Boolean predicates of selected K_5 edges. This is essentially a 10-state constraint satisfaction problem (CSP). However, due to the set of port types and multiplicities, the constraints may force the vertices in a block to select the same K_5 edge, up to sign. This leads to thinking about the block-flip setup and how it might influence the difference vectors with each flip.

Exploring difference solutions in secant assignments

Each block corresponds to "compatible secant assignments" that satisfy certain homogeneous equations. A nonzero difference solution is one of these assignments. The space of all these differences can be computed by counting the dimension of compatible secant assignments, which might be solvable through the 10-state difference CSP. The space itself is a linear F_2 space.

Reconstructing blocks through system of equations

To reconstruct blocks, I could use the graph of constraints and apply the "difference SAT" concept on each label pair. If we don't know the labels, we might calculate the solution directly by solving a system of equations. Each vertex's incident ports would produce bits that must connect through edges.

Solving with linear algebra and matroids

Each edge has a bit, and incident edge bits realize one of 10 patterns. This suggests a standard constraint satisfaction problem with a domain of 11 patterns. I wonder if I can solve this using linear algebra since these patterns form the cocycle matroid of K_5 . The global difference space might correspond to a binary matroid, which could be computed via rank calculations. I'll simplify the 56 ports to 16 unique types.

Analyzing edge incidences and classifications

We're considering how the chosen K5 edges intersect with cuts, requiring equal incidences for pairs of edges. This leads to a binary relation between edges, with specific conditions for edge crossings with certain cuts. The domain involves 10 K5 edges and 15 cuts, but cuts are constrained to even subsets. So, there are 16 unique cut types, including the empty cut.

Considering relations and potential graph isomorphism

So the 4-subsets behave the same as singleton cuts, meaning port types map to all cuts in K5, including their complements and the empty cut. This leads to relations that might resemble a graph isomorphism problem. The structure suggests a potential CSP or even a 2-SAT/union-find scenario, though I need to pause and continue researching.

Investigating path consistency and counterexamples

We started with the solution set Ω in binary encoding. Path consistency (PC) works well when pairwise projections are full or parity relations, but I wonder: will PC reduce to the correct closure starting from the original 5 relations? I'll try generating EO instances and test PC closure to check for potential counterexamples.

Testing path consistency on small EO networks

I'll generate EO instances for small n (around 2-5) by pairing 56 port types and creating random perfect matchings of half-edges. The CSP will track the intersection of relations between nodes. I'll apply path consistency (PC) to see if it matches the expected results, and check for counterexamples.

Implementing relations and path consistency

I'll represent relations as 5x5 boolean matrices. For multiple edges between the same pair, I need to consider intersections. If the edge is undirected, the relation between vertices is based on port types. For loops, the relation is unary, and I'll reduce it to a unary domain. I'll then apply path consistency to handle binary CSP with unary domains. Time to write the closure algorithm!

Implementing path consistency for binary CSP

Code:

```
import itertools, random, math, collections, functools, time, sys, statistics
labels = [1,2,3,4,5]
# all port types expanded
port_types = [frozenset()] + [frozenset(S) for S in itertools.combinations(labels,2) for _ in range(4)] + [frozenset(set(labels)-{j}) for j in labels for _ in range(3)]
len(port_types)
```

Output:

```
56
```

Implementing path consistency with relation matrices

Code:

```
# unique relation matrices
def R(S,T):
    return {(x,y) for x in labels for y in labels if ((x in S) ^ (y in T))}
# precompute
relation_cache={}
for S in set(port_types):
    for T in set(port_types):
        relation_cache[(S,T)] = R(S,T)
len(relation_cache)
```

Output:

256

Creating random instance and exact solver

I need to generate a random instance of the problem and create an exact solver for it. This will let me test the effectiveness of the solution and validate the approach. Once I have the instance, I can work on solving it step by step.

Generating random instance and pairings

Code:

```
def random_pairing(items):
    items=list(items)
    random.shuffle(items)
    return [(items[i], items[i+1]) for i in range(0,len(items),2)]

def make_random_instance(n, allow_loops=True, seed=None):
    if seed is not None:
        random.seed(seed)
    ports=[(v,i,port_types[i]) for v in range(n) for i in range(56)]
    if allow_loops:
        pairs=random_pairing(ports)
    else:
        # pair ports from different vertices greedily; simple method maybe fail
        pools={v:[(v,i,port_types[i]) for i in range(56)] for v in range(n)}
        verts=[v for v in range(n) for _ in range(56)]
        # attempt using configuration model on stubs
```

Searching for PC exactness counterexamples

Code:

```
def pc_exact(inst):
    sols=exact_solutions(inst)
```

```

tu,tb=true_projections(sols, inst["n"])
pu,pb=path_consistency(inst)
return tu==pu and tb==pb, sols, tu,tb,pu,pb

random.seed(0)
counter=0
for n in range(2,7):
    found=False
    for t in range(200):
        inst=make_random_instance(n, allow_loops=True)
        ok, sols, tu,tb,pu,pb = pc_exact(inst)
        counter += 1
        if not ok:
            print("counterexample n",n,"trial",t,"#sol",len(sols))
            # summarize diff
            for v in range(n):
                if tu[v]!=pu[v]:
                    print(" unary",v,tu[v],pu[v])
            diffpairs=[]
            for k in tb:
                if tb[k]!=pb[k]:
                    diffpairs.append(k)
            print("num diffpairs",len(diffpairs), diffpairs[:10])
            found=True
            bad_inst=inst; bad_data=(sols,tu,tb,pu,pb)
            break
    if found:
        break
print("tested",counter)

```

Output:

```

counterexample n 3 trial 3 #sol 0
 unary 1 set() {5}
num diffpairs 0 []
tested 204

```

Investigating unsat detection and improving PC

I found an unsat instance where the path consistency (PC) algorithm didn't detect it. Maybe loops are causing weird issues, and I could try removing them. It's possible our PC algorithm needs better unary support, but it might still work. I'll search for a satisfiable counterexample to test PC's limitations.

Detecting unsatisfiable instances in PC algorithm

Code:

```

random.seed(1)
found=False
for n in range(2,7):

```

```

for t in range(1000):
    inst=make_random_instance(n, allow_loops=True)
    sols=exact_solutions(inst)
    if not sols:
        continue
    ok, sols, tu,tb,pu,pb = pc_exact(inst)
    if not ok:
        print("SAT counterexample n",n,"trial",t,"#sol",len(sols))
        for v in range(n):
            if tu[v]!=pu[v]:
                print(" unary",v,tu[v],pu[v])
        diffpairs=[k for k in tb if tb[k]!=pb[k]]
        print("num diffpairs",len(diffpairs), diffpairs[:10])
        bad_inst=inst; bad_data=(sols,tu,tb,pu,pb)
        found=True
        break
    if found:
        break
print("done")

```

Output:

```
done
```

Exploring satisfiability and detecting unsat

I didn't find a satisfiable counterexample in 1,000 random trials with loops, so I might need to expand my search, focusing on no-loops and more adversarial cases. For detecting unsat, the algorithm might need to find a solution first. I wonder if combining PC with satisfiability checks could work.

Searching satisfiability with no-loop constraints

Code:

```

random.seed(2)
# search no-loop, n even maybe easier to pair without loops for n>=2
def search_pc(num_trials=500, ns=range(2,7)):
    sat_bad=None; unsat_bad=None
    for n in ns:
        if (56*n)%2:
            continue
        for t in range(num_trials):
            try:
                inst=make_random_instance(n, allow_loops=False)
            except:
                continue
            ok, sols, tu,tb,pu,pb = pc_exact(inst)
            if not ok:
                if sols and sat_bad is None:

```



```

        sat_bad=(n,t,inst,sols,tu,tb,pu,pb); print("SAT bad",n,t,len(sols)); return
sat_bad,unsat_bad
        if not sols and unsat_bad is None:

```

Output:

```

n 2 done
n 3 done
n 4 done

```

Considering trial limitations due to environment constraints

It seems like there may have been a timeout issue in Python due to an automatic 60-second interrupt, possibly caused by the environment settings. It might be better to continue with fewer trials to avoid hitting that limit and ensure more controlled testing moving forward.

Reducing trials and handling interruptions

Code:

```

# continue shorter
random.seed(3)
sat_bad=None; unsat_bad=None
for n in [5,6]:
    count=0
    trials=60
    for t in range(trials):
        try:
            inst=make_random_instance(n, allow_loops=False)
        except:
            continue
        count += 1
        ok, sols, tu,tb,pu,pb = pc_exact(inst)
        if not ok:
            if sols:
                sat_bad=(n,t,len(sols)); print("SAT bad",sat_bad); raise SystemExit
            else:
                unsat_bad=(n,t); print("UNSAT bad",unsat_bad); raise SystemExit
    print("n",n,"tested",count)
print("done")

```

Output:

```

n 5 tested 0
n 6 tested 0
done

```

Analyzing no-loop generation and unsat detection issues

The no-loop generation failed for $n=5,6$ due to pairing limitations. I could improve the pairing with a multigraph approach later. For satisfiable instances, PC is working, but unsat detection isn't. Maybe using singleton-PC could help, or a separate SAT algorithm. I can test PC with no-loop random generation.

Exploring multigraph construction and random pairing

For small n and degree 56, I can use the configuration model by pairing vertices. For $n=5$, a simple graph isn't possible, but a multigraph works. I could randomly match stubs by using a derangement algorithm with rejection, though it may fail for some n . Random selection could help maintain counts.

Implementing pairing for multigraph construction

This approach should work as long as we have two nonzero vertices, though it can get stuck when there's only one left with stubs. Using a Havel-Hakimi-style approach might help. For regular degrees in the multigraph, a greedy pairing of stubs could work. I'll try implementing the algorithm with backtracking.

Implementing no-loop random graph generation

Code:

```
def make_random_instance_noloops(n, seed=None):
    if seed is not None:
        random.seed(seed)
    stubs={v:[(v,i,port_types[i]) for i in range(56)] for v in range(n)}
    vertices=list(range(n))
    pairs=[]
    # Use randomized pairing with backtracking restarts
    for attempt in range(100):
        stubs={v:[(v,i,port_types[i]) for i in range(56)] for v in range(n)}
        pairs=[]
        success=True
        while True:
            nonzero=[v for v in vertices if stubs[v]]
            if not nonzero:
                break
            if len(nonzero)==1:
                success=False; break
            # choose first vertex with
```

Output:

```
(140, False, [56, 56, 56, 56, 56])
```

Searching for no-loop counterexamples

Okay, now I need to search for counterexamples where no loops are allowed. This will help test whether the approach works without looping, and I'll look for edge cases or specifics to refine the current method. Let's go ahead and see if we can find any applicable examples.

Searching no-loop counterexamples

Code:

```
random.seed(4)
sat_bad=None; unsat_bad=None
for n in range(2,8):
    for t in range(200):
        inst=make_random_instance_noloops(n)
        ok, sols, tu,tb,pu,pb = pc_exact(inst)
        if not ok:
            if sols:
                sat_bad=(n,t,len(sols)); print("SAT bad",sat_bad)
                bad_inst=inst; bad_data=(sols,tu,tb,pu,pb)
                raise SystemExit
            else:
                if unsat_bad is None:
                    unsat_bad=(n,t); print("UNSAT bad",unsat_bad)
                    bad_unsat=(inst,sols,tu,tb,pu,pb)
                    # continue to search sat bad
        print("n",n,"done")
print("no sat bad found",unsat_bad)
```

Output:

```
UNSAT bad (2, 0)
n 2 done
n 3 done
n 4 done
n 5 done
n 6 done
n 7 done
no sat bad found (2, 0)
```

Inspecting unsat counterexample structure

I need to check the structure of the unsatisfiable no-loop example. More importantly, testing up to n=7 with 200 trials each suggests that PC exactness works for satisfiable cases, but not for unsatisfiable ones. Maybe there's a way to detect unsat by checking for any solution using decision algorithms on the reduced network.

Inspecting unsat examples for PC closure behavior

It might be that, after applying PC closure and getting a block structure, satisfiability boils down to consistency with the parity equations. For unsat instances, PC closure might yield unary domains that have values greater than 0, or pairwise relations that don't follow the expected pattern. I'll need to examine these unsat examples.

Inspecting unsatisfiable examples and unary sizes

Code:

```
inst,sols,tu,tb,pu,pb = bad_unsat
print("n",inst['n'], "sols",len(sols))
print("unary sizes PC",[len(pu[v]) for v in range(inst['n'])])
# summarize binary relation sizes
sizes=collections.Counter(len(pb[k]) for k in pb)
sizes
```

Output 1:

```
n 2 sols 0
unary sizes PC [0, 5]
```

Output 2:

```
Counter({0: 2})
```

Investigating bad_unsat inconsistency

It seems like the "bad_unsat" example might actually be detecting path consistency when it shows a unary size of 0. But there's a discrepancy in the projection. The unary values differ between `pc_exact` and `pc` — one has an empty set while the other has {5}. I need to inspect this further.

Finding concrete unsat counterexample with nonempty PC

Code:

```
# Let's find a concrete unsat no-loop counterexample with nonempty PC
random.seed(5)
bad_unsat=None
for n in range(2,6):
    for t in range(100):
        inst=make_random_instance_noloops(n)
        sols=exact_solutions(inst)
        pu,pb=path_consistency(inst)
        if not sols and all(pu[v] for v in range(n)):
            bad_unsat=(n,t,inst,sols,pu,pb)
            print("found unsat nonempty PC", n,t)
            raise SystemExit
print("none")
```

Output:

```
none
```

Exploring unsat detection with path consistency

The search found no unsat instances with non-empty unary conditions for $n < 6$ in no-loop cases. However, previous "unsat_bad" instances had an empty projection for all variables. I suspect PC might detect unsat if any unary is empty, even though some variables may have non-empty unary. I need to verify.

Searching for unsat with nonempty unary constraints

Code:

```
# search more unsat nonempty PC for n up to 7 with moderate trials
random.seed(6)
for n in range(2,8):
    cnt=0
    found=False
    for t in range(300):
        inst=make_random_instance_noloops(n)
        pu,pb=path_consistency(inst)
        if any(len(pu[v])==0 for v in range(n)):
            continue
        sols=exact_solutions(inst)
        if not sols:
            print("unsat nonempty PC found n",n,"trial",t)
            found=True
            break
        cnt += 1
    print("n",n,"tested candidate sat/nontrivial",cnt, "found",found)
    if found: break
```

Output:

```
n 2 tested candidate sat/nontrivial 0 found False
n 3 tested candidate sat/nontrivial 0 found False
n 4 tested candidate sat/nontrivial 0 found False
n 5 tested candidate sat/nontrivial 0 found False
n 6 tested candidate sat/nontrivial 0 found False
n 7 tested candidate sat/nontrivial 0 found False
```

Testing planted satisfiable instances for PC exactness

It seems all nonempty PC instances are satisfiable, but the candidate counts are still zero. Maybe this is because PC always creates empty solutions, or "exact_solutions" is too expensive. To fix this, I need to generate planted satisfiable no-loop examples and test PC's exactness using a compatibility graph approach.

Generating planted satisfiable instances with no loops

Code:

```
def make_planted_instance_noloops(labels_assign, max_attempts=100):
    n=len(labels_assign)
    for attempt in range(max_attempts):
        stubs={v:[(v,i,port_types[i]) for i in range(56)] for v in range(n)}
        pairs=[]
        success=True
        while True:
            nonzero=[v for v in range(n) if stubs[v]]
            if not nonzero:
                break
            if len(nonzero)==1:
                success=False; break
            # choose vertex with few compatible partners? approximate by max stubs
            u=max(nonzero, key=lambda v: len(stubs[v]))
            # build candidate partner stubs from other vertices compatible with some stub of u
            # choose random stub of u with at least one compatible partner
            random.shuffle(stubs[u])
            found=False
            for idx,su in enumerate(stubs[u]):
                xu=labels_assign[u]
                comp=[]
                for v in nonzero:
                    if v==u: continue
                    xv=labels_assign[v]
                    for j,sv in enumerate(stubs[v]):
                        if ((xu in su[2]) ^ (xv in sv[2])): # satisfied edge
                            comp.append((v,j,sv))
                    if comp:
                        pass
            if comp:
                found=True
                # choose one random compatible partner
```

```

        v,j,sv=random.choice(comp)
        stubs[u].pop(idx)
        stubs[v].pop(j)
        pairs.append((su,sv))
        break
    if not found:
        success=False; break
    if success:
        unary={v:set(labels) for v in range(n)}
        binary={(u,v): {(x,y) for x in labels for y in labels} for u in range(n) for v in
range(n) if u!=v}
        edges=[]
        for (u,iu,S),(v,iv,T) in pairs:
            rel=relation_cache[(S,T)]
            binary[(u,v)] &= rel
            binary[(v,u)] &= {(y,x) for x,y in rel}
            edges.append((u,v,S,T))
        return {"n":n,"unary":unary,"binary":binary,"edges":edges,"loops":[]}
    raise RuntimeError("failed planted")
# test
random.seed(0)
inst=make_planted_instance_noloops([1,2,3,4])
sols=exact_solutions(inst)
len(sols), [1,2,3,4] in sols[:10], sols[:5]

```

Output:

```
(1, False, [(1, 2, 3, 4)])
```

Testing PC exactness on planted satisfiable instances

I plan to test the PC exactness on a variety of planted satisfiable instances. By doing so, I'll be able to better understand how well the consistency checks hold up when we know the instances should be satisfiable. This will help refine my approach.

Testing PC exact on planted SAT instances

Code:

```

random.seed(7)
sat_bad=None
stats=collections.Counter()
for n in range(2,8):
    for t in range(100):
        x=[random.choice(labels) for _ in range(n)]
        inst=make_planted_instance_noloops(x)
        sols=exact_solutions(inst)
        ok, sols, tu,tb,pu,pb = pc_exact(inst)
        stats[(n,len(sols))]+=1
    if not ok:

```

```

print("SAT bad planted n",n,"trial",t,"#sol",len(sols),"x",x)
for v in range(n):
    if tu[v]!=pu[v]:
        print(" unary",v,tu[v],pu[v])
diffpairs=[k for k in tb if tb[k]!=pb[k]]
print("num diffpairs",len(diffpairs), diffpairs[:10])
sat_bad=(inst,sols,tu,tb,pu,pb,x)
raise SystemExit
print("no sat bad", stats.most_common(20)[:10])

```

Output:

```

no sat bad [((2, 1), 100), ((3, 1), 100), ((4, 1), 100), ((5, 1), 100), ((6, 1), 100), ((7, 1),
100)]

```

Generating multi-solution planted instances

My planted generator tends to produce a unique solution due to its greedy approach. To create multi-solution cases, I could try constructing block flips and selecting edges that satisfy both solutions. This should help generate instances with multiple valid solutions. I'll work on implementing this idea.

Generating multi-solution planted instance

Code:

```

def make_planted_instance_from_solutions(sol_list, max_attempts=200):
    n=len(sol_list[0])
    for attempt in range(max_attempts):
        stubs={v:[(v,i,port_types[i]) for i in range(56)] for v in range(n)}
        pairs=[]
        success=True
        while True:
            nonzero=[v for v in range(n) if stubs[v]]
            if not nonzero:
                break
            if len(nonzero)==1:
                success=False; break
            u=max(nonzero, key=lambda v: len(stubs[v]))
            random.shuffle(stubs[u])
            found=False
            for idx,su in enumerate(stubs[u]):
                comp=[]
                for v in nonzero:

```

Output:

```

(2, [(1, 1, 1, 1), (2, 2, 2, 2)])

```


Testing PC exactness on multi-solution cases

I'll test the PC exact method on the multi-solution instances I've generated. This will help me see if it handles multiple valid solutions correctly and whether everything works as expected. Looking forward to checking if this approach is more effective with various solutions in place!

Searching for multi-solution planted configurations

Code:

```
random.seed(8)
for k in [2,4]:
    for n in [4,5,6]:
        for t in range(50):
            # choose k solutions with intended block structure maybe
            base=[random.choice(labels) for _ in range(n)]
            sols_target=[tuple(base)]
            # generate up to 3 extra solutions by flipping random vertices to second labels?
            # easier choose another solution and include xor closure? but domain not binary
            if k>=2:
                alt=[random.choice([x, random.choice([y for y in labels if y!=x]])] for x in base]
                sols_target.append(tuple(alt))
            if k>=4:
                # choose second independent
```

Output:

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[28], line 24
     22 sols_target=list(dict.fromkeys(sols_target))
     23 try:
--> 24     inst=make_planted_instance_from_solutions(sols_target)
     25 except RuntimeError:
     26     continue

Cell In[27], line 25, in make_planted_instance_from_solutions(sol_list, max_attempts)
     23         if not (((sol[u] in su[2]) ^ (sol[v] in sv[2]))):
     24             ok=False; break
--> 25         if ok:
     26             comp.append((v,j,sv))
     27 if comp:

KeyboardInterrupt:
```

Exploring path consistency and decomposability impact

I'm working on continuing the research to solve the problem. Maybe I can derive a formal proof of block-flip based on majority/minority closure, which we have already explored. If we break down vertices to a smaller size (≤ 2), we should get a majority operation that preserves consistency. Could this lead to a solution?

Linking binary projections and path consistency

The actual solution set consists of tuples consistent with the true binary projections. Path consistency computes the smallest set of relations that includes the original constraints and is closed under composition. I need to show that binary projections are contained in the PC closure, ensuring the solution set.

Analyzing path consistency and variable domains

I'm considering if the pair relations for values consistent with F_u, F_v are supersets of the true binary projections. To ensure no impossible extra pairs, this might rely on the path consistency enforced by multiset paths between port-types. I'll analyze the local structure of vertices and their neighbors to test this idea.

Considering path consistency through cycles and multiplicities

I'm wondering if path consistency can be inferred from combinations of constraints, especially around cycles. The relations involving neighboring variables should restrict the domain of vertex v . I might explore alternative approaches, like dynamic polynomial algebra over simplex vectors, to compute counts.

Exploring low-degree polynomials and transfer matrices

I'm considering low-degree polynomials, especially since each simplex row has only five labels. Is there an approach using transfer matrices over five states? The idea involves expanding over edge subsets, but could the formula use matrix rank somehow? I'm looking at how to compute the partition function Z using Fourier transforms over syndromes.

Exploring decomposition of $w(r)$ and Fourier transforms

I'm thinking about whether $w(r)$ can be expressed as a combination of delta functions of linear constraints, which might let me decompose it into affine signatures. Since w has 16 values, its Fourier transform might be a 5-point indicator. Decomposing it could lead to a more efficient algorithm.

Exploring factorization of $w(r)$ and quadratic equations

Since 5 isn't a power of 2, I might need several subspaces. If the global count is a linear combination of affine systems, the complexity could get too high unless there's a special factorization across vertices. But maybe $w(r)$ could be factored as a determinant, or I could use quadratic equations for counting solutions over affine space. This approach looks promising.

Outlining the problem setup

The problem is to count assignments $z_v \in F_2^4$

Exploring quadratic equations and propagation

Counting solutions to the mixed linear and quadratic equations over F_2 could be hard in general, but the projective structure and the fact that the quadratic form is the same at each vertex may simplify things. I wonder if homogenization could help, although counting common zeros of quadratic polynomials remains non-trivial. Maybe the linear system could propagate the quadratic constraints. I'll need to check that.

Exploring secant vectors and relations

For different solutions, I'm considering whether secant vectors d could satisfy $q(d) = 1$. The original vectors in G are singular, nonzero, and might not directly relate to q values unless I use projective geometry. Is there a way to compute relations between vertices involving the bilinear form B ? The idea is to find ways to express relations algebraically and possibly normalize them.

Exploring the partition sizes based on weight

I'm considering the partitioning of G based on x mapped to $a \cdot x$. If a has weight 1 or 2, the size of the set S_a (which consists of elements g_i where $a \cdot g_i = 1$) will vary — it could be 2 for some a , 4 for others, and 0 for $a = 0$. It seems the weight of a might determine the size of S_a , so I want to check that.

Computing size of S based on vector weight

Code:

```
all_a = allvec
size_by_w=collections.defaultdict(set)
for a in all_a:
    S=[i for i,g in G.items() if sum(ai*gi for ai,gi in zip(a,g))%2==1]
    size_by_w[sum(a)].add(len(S))
size_by_w
```

Output:

```
defaultdict(set, {0: {0}, 1: {2}, 2: {2}, 3: {4}, 4: {4}})
```

Exploring the quadratic form for subsets

Nonzero a with weight 1 or 2 correspond to 2-subsets, while weight 3 or 4 correspond to 4-subsets. This is good. Now, I think the quadratic form $q(a)$ might indicate the size, so I need to compute $q(a)$ for weight $1/2$ to check if this holds up.

Computing quadratic form for weight $1/2$

Code:

```
def q(v):
    return sum(v[i]*v[j] for i in range(4) for j in range(i+1,4))%2
vals=collections.defaultdict(list)
for a in all_a:
    if a==(0,0,0,0): continue
    vals[(sum(a),q(a))].append(a)
vals.keys(), {k:len(v) for k,v in vals.items()}
```

Output:

```
(dict_keys([(1, 0), (2, 1), (3, 1), (4, 0)]),
 {(1, 0): 4, (2, 1): 6, (3, 1): 4, (4, 0): 1})
```

Exploring relations, isometries, and group actions on subsets

I'm wondering if relation types correspond to linear functionals with certain parities or quadratic types. Could there be isometries that map nonzero a to weight 1 or 3 depending on the quadratic form q ? The group might act transitively on subsets of size 2 or 4. I need to check if the S_5 group can preserve the quadratic form q and if different weights correspond to distinct subset sizes.

Considering affine transformations, solution sets, and code dimensions

I'm thinking about choosing local affine transformations mapping a two-label domain to $\{0,1\}$. The idea could be to compute these domains and try a counterexample to the block-flip theorem. To be confident, I think the count might be 2^k , where k is the dimension tied to the linear span of the difference solutions.

Exploring difference CSP and semilattice operations

I'm considering whether the difference CSP is exact. To test, I'd first find a base solution and then count other independent difference solutions. The basic relation here involves the condition $a \cdot d_u = b \cdot d_v$. It seems the domain has an 11-element size — possibly representing edges in a complete graph.

Examining codewords and partial cubes in K_5

The codewords $C(e)$ are length-15 cut incidence vectors from K_5 edges, similar to a graphic matroid. The 11-element domain might have a fixed majority operation, with relations corresponding to bit-equality under 15 predicates. This setup might define a partial cube, possibly embedding K_5 's edge graph into a hypercube.

Simplifying difference CSP with linear functionals

The difference CSP can be framed using even subsets S and their characteristic vectors over F_2^5 . Each port type S corresponds to a linear functional $x \mapsto \chi_S \cdot x$. The problem reduces to finding assignments in the 11-element domain E , where relations are quadratic equalities. This formulation might be easier due to the structure of the domain.

Exploring block-flip structure and solution count

The block-flip structure in the original solutions might correspond to selecting independent atoms in the difference system. The set E consists of weight-0 or weight-2 vectors in F_2^5 , totaling 11 elements. A simpler majority/minority theorem might be possible. The solution count could potentially be derived by checking assignments satisfying linear equations in E .

Considering local secant sums and differences in block-flips

At each vertex, the sum of two local secants could result in another secant or zero, depending on whether the block supports are disjoint. The difference system isn't necessarily closed and may not always have path consistency. However, I wonder if leveraging the set E of weight-0/2 vectors could help resolve this.

Exploring properties of affine subspaces intersecting E

Intersecting affine subspaces with E might reveal special properties that simplify the problem. It looks like E contains weight-0 and weight-2 vectors from the 4D even-weight subspace of F_2^5 , which gives us 11 distinct points. This is similar to the Johnson scheme $J(5,2)$. There's also an alternative approach framing satisfiability using bit assignments on connected components.

Formulating the problem with bit-level constraints

The 16-bit pattern for each vertex type is governed by specific equality or complementation constraints. If multiple ports of the same type go to different neighbors, the edge constraints force certain bits to be equal or complementary. Defining the bit variables $y_{v,S}$ for each vertex-type occurrence simplifies this into a graph with type-variables, where solving the problem is reduced to managing affine relationships between bits.

Simplifying Boolean variables with connectivity constraints

At each vertex, the 16 bits must form one of five specific patterns. By defining one Boolean variable for each connected component in the half-edge graph, we can reduce complexity. While two half-edge positions connected by an edge enforce specific bit relationships, the shared bit across ports of the same type simplifies the system by collapsing to type-variables $y_{v,S}$.

Examining the algebraic structure of (Y) and finding simplifications

I'm exploring the algebraic structure of Y , a 5-element subset of $\{0, 1\}^{16}$, which represents row patterns over all 16 even subsets. It's possible that the allowed 16-bit vectors form an affine subspace, and I wonder if simplifying the system with Gaussian elimination could help. I'm thinking about applying algebraic proofs, like leveraging the Helly property of cuts.

Analyzing the algebra of relations and exploring path consistency

I'm focusing on the algebra of unary and binary relations, trying to compute their closure under operations like intersection and composition. If I apply these operations to raw relations, I might find patterns that reveal how the path consistency algorithm operates. This could lead to a more direct solution. Let's try computing it using Python.

Computing closure of binary relations under operations

Code:

```
# compute closure of raw binary relations under intersection, converse, composition and unary restrictions?
D=labels
all_pairs={(x,y) for x in D for y in D}
raw_rels=set()
for S in set(port_types):
    for T in set(port_types):
        raw_rels.add(frozenset(relation_cache[(S,T)]))
len(raw_rels)
```

Output:

256

Considering the feasibility of computing closure

I'm thinking that computing the closure could be manageable. I'll explore it further by applying the necessary operations and see if the size is within a reasonable scope. This could help ensure I approach the problem effectively. Let's proceed with trying to compute it.

Computing relation closure from pairs

Code:

```
# Represent relations as frozensets of 25 pair indices maybe faster
pair_to_idx={(x,y):i for i,(x,y) in enumerate(all_pairs)}
idx_to_pair={i:p for p,i in pair_to_idx.items()}
def rel_from_pairs(pairs): return frozenset(pair_to_idx[p] for p in pairs)
def pairs_from_rel(r): return {idx_to_pair[i] for i in r}
raw = {rel_from_pairs(relation_cache[(S,T)]) for S in set(port_types) for T in set(port_types)}
len(raw)
```

Output:

256

Exploring closure computation possibilities

I'm thinking of continuing with closure computation, keeping in mind that the upper limit could be something like 1024. I'll need to check if that number is feasible for the calculations or if additional adjustments are needed. Let's move forward and examine the possibilities.

Computing closure under relations and composition

Code:

```
full_rel=rel_from_pairs(all_pairs)
id_rel=rel_from_pairs({(x,x) for x in D})

def converse(r):
    return frozenset(pair_to_idx[(idx_to_pair[i][1], idx_to_pair[i][0])] for i in r)

# Precompute matrices maybe
rel_list=list(raw|{full_rel,id_rel})
# composition helper
def compose(r,s):
    R=pairs_from_rel(r); S=pairs_from_rel(s)
    comp={(x,z) for (x,y) in R for (y2,z) in S if y==y2}
    return rel_from_pairs(comp)
def proj1(r):
    return frozenset(x for x,y in pairs_from_rel(r))
def proj2(r):
    return frozenset(y for x,y in pairs_from_rel(r))

# closure under intersection, composition, converse
closure=set(rel_list)
changed=True
iters=
```

Output 1:

```
iter 1 size 29498 changed True
```

Output 2:

```
29498
```

Considering closure and searching for combinations

The closure is huge, and not easy to compute. Maybe I should focus on unary relations first, checking if all 32 unary subsets can appear. Binary closure, however, is still massive. I could look for counterexamples systematically using SAT/SMT for smaller values of n , like 3 or 4. Let's explore combining raw relations.

Exploring intersection closure and pair relations

Multiple edges between the same pairs intersect. Since there are 256 raw relations, arbitrary intersections might produce manageable sets. An EO instance on n vertices may be an $n \times n$ matrix with such relations plus feasibility conditions. However, not every pair relation is realizable due to vertex constraints. Let's compute the intersection closure of raw relations.

Computing intersection closure of raw relations

Code:

```
raw_list=list(raw)
inter_closure={full_rel}
# meet closure by all raw relations sequentially: intersections of subsets
# but there are many. Use iterative set meet.
changed=True
while changed:
    changed=False
    cur=list(inter_closure)
    for r in cur:
        for s in raw_list:
            t=r & s
            if t not in inter_closure:
                inter_closure.add(t); changed=True
    print(len(inter_closure), changed)
len(inter_closure)
```

Output 1:

```
257 True
27737 True
46037 True
46037 False
```

Output 2:

```
46037
```

Continuing research into EO and path consistency (PC)

Searching for a SAT PC counterexample with $n=3$ using realizable pair relations seems challenging. However, we've found no counterexamples to PC exactness for loopless EO instances after extensive search. We should focus on proving PC exactness for loopless EO. It's unclear whether loops are allowed in EO; we need to check.

Refining approach to majority operation and EO instances

It seems that multigraphs often allow loops. However, maybe the exceptional case in the EO dichotomy involves simple signature graphs without loops. I need to consider this conditional on the usual loopless convention. I could focus on proving PC exactness for loopless instances using strict width 2 from majority operations.

Generating relations for majority operation in satisfiable instances

For random satisfiable instances, I'll generate relations and focus on finding a majority operation that preserves them. Since there are finitely many relations and potentially simple pair projections, I'll inspect if the projections are full, singletons, or rectangles. The user wants progress on f56, so partial results work here.

Collecting binary pair relations from SAT instances

Code:

```
# gather true binary projections from satisfiable planted and random instances
def collect_pair_relations(samples=100):
    rels=set()
    # planted unique
    for n in [2,3,4,5]:
        for _ in range(samples):
            x=[random.choice(labels) for _ in range(n)]
            inst=make_planted_instance_noloops(x)
            sols=exact_solutions(inst)
            _,tb=true_projections(sols,n)
            rels |= set(map(frozenset, tb.values()))
    return rels
random.seed(9)
rels=collect_pair_relations(30)
len(rels)
```

Output:

```
25
```

Inspecting relation types and sizes

I should take a look at the different types of relations that arise and evaluate their sizes. This will help me figure out what I'm working with, especially as I work through random satisfiable instances and their properties. The size of these relations could be important for next steps.

Inspecting relation type sizes

Code:

```
sorted(collections.Counter(len(r) for r in rels).items())
```

Output:

```
[(1, 25)]
```

Generating two-solution instances

I need to make sure I'm collecting relation data from multi-solution instances, not just the unique solutions that create singleton relations. It might be helpful to intentionally generate instances with exactly two solutions. We have a generator for that, so let's use simpler, predetermined two-solution cases across n .

Generating and collecting multi-solution instances

Code:

```
def relation_from_pairs_set(pairs):
    return frozenset(pair_to_idx[p] for p in pairs)
# generate some 2-solution instances
multi_rels=set()
random.seed(10)
for n in [2,3,4,5]:
    for t in range(50):
        base=[random.choice(labels) for _ in range(n)]
        alt=base[:]
        changed=False
        for i in range(n):
            if random.random()<0.5:
                choices=[y for y in labels if y!=base[i]]
                alt[i]=random.choice(choices); changed=True
        if not changed:
            alt[0]=random.choice([y for y in labels if y!=base[0]])
        try:
            inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)], max_attempts=50
```

Output 1:

```
num rels 307 sizes [(1, 25), (2, 282)]
```

Output 2:

```
-----
KeyError                                Traceback (most recent call last)
Cell In[37], line 29
    27 by_size=collections.defaultdict(list)
    28 for r in multi_rels:
--> 29     by_size[len(r)].append({idx_to_pair[i] for i in r})
    30 for s in sorted(by_size)[:10]:
    31     print("size",s,"example",by_size[s][0])

KeyError: (2, 4)
```

Considering next steps for solving f56

I'm wondering if any variables from previous attempts are still present—though the developer says not to assume anything. I may rerun the process to be sure. The user wants to continue tackling f56, so I'm thinking of combining PC and SAT via a greedy self-reduction algorithm. If it gets stuck, we can detect unsatisfiability by contradiction.

Testing PC effectiveness and considering proof of completeness

I'm thinking, if PC is nonempty but the instance is unsatisfiable, maybe under loopless conditions this doesn't happen. That suggests PC might be a full decision procedure for loopless instances. I could test more random no-loop instances for robustness, but exact solving is expensive. For unsat checks, we might be able to rely on a SAT solver. I also want to attempt a proof of PC completeness using majority/minority block structures and loop absence.

Exploring algorithm for loopless instances and constructing solutions

If the algorithm works for loopless instances, can we prove it? Since PC closure is true for loopless instances, I might consider constructing a solution starting with a unary domain assignment. I'm thinking about using the block-flip theorem, which could help in generating a solution. If each assignment extends globally, we can build a solution greedily based on strict width 2. We'll see if we can identify the correct majority polymorphism for these relations.

Exploring relation preservation and alternative approaches

It seems that pairwise projections are preserved with a conservative majority, and the Barto-Kozik theorem might hold here. However, we need to examine whether raw relations in the loopless EO are preserved. Alternatively, spectral graph theory could be a promising track. Using rank-2 matrices for edge constraints and considering the partition function on a multigraph could reduce complexity. I'll need to experiment with this formulation.

Investigating local weight and transformation properties

It looks like $w(r)$, expressed as a sum of weighted characters, might fit into the structure of matchgate identities or free-fermion conditions. However, dealing with high-arity cases like 56 might make things complex. I think I could compute whether $w(r)$ satisfies conditions like affine signatures. Given its nonzero nature, it's unlikely though. I'll need to explore rank-based formulations using incidence matrices for a more exact result.

Exploring the partition function and its connection with matroids

I wonder if the partition function could be evaluated as the Tutte polynomial of a matroid based on the edge-port incidence matrix, where the weight function possibly takes values like 1 or -3 based on certain conditions. This resembles the enumeration of a linear code with a 3-valued weight. Could Greene's theorem help here if the weight is related to a quadratic form?

Analyzing weight values and group symmetry

It seems that under different scenarios, the weight function assigns different values (1 or -3) based on the vector's weight. The challenge is that weights 1, 2, 3, and 4 are distinguished, but the symmetry of the group isn't fully preserved under the full S5 group. However, with a chosen basis, certain vector behavior becomes singled out.

Examining port types and symmetry

Okay, so r_v seems to map to an even subset T_v , which represents the set of labels toggled by the syndrome—nice! And $w(r_v) = 5 - 2|T_v|$, where the weight depends on the size of this even subset. It looks like the dual model weight function is now more symmetric and combinatorial. Interesting.

Dual model formulation and next steps

The dual model sums over subsets of edges with a weight formula that involves symmetric differences. This is essentially a flow model involving group C_2^4 , and it has a local linear weight function based on even subset sizes. For further work, I can either propose a stronger theoretical result or share detailed experimental progress. I'll remain transparent and focus on providing a more substantive update.

Exploring a solution for fibers recovery

I'm considering the subdirect product of 2-element affine algebras, applying the Bulatov-Dalmau/Feder-Vardi strict width 2 result for PC exactness. I think the key step is defining a fixed majority polymorphism on the induced binary template. I want to use our findings and universal algebra theorems to help recover fibers via path consistency.

Testing path consistency on loopless PC-closed networks

I'm thinking that path consistency might possibly eliminate domains with sizes larger than 2 in a PC-closed network with nonempty domains. I'm planning to test this experimentally by analyzing random satisfiable PC-closed networks. I'll search for instances with domain size 3 and check if they survive after path consistency.

Testing path consistency on no-loop instances

Code:

```
# search satisfiable no-loop instances with PC unary domain >2
random.seed(11)
found=None
for n in range(2,8):
    for t in range(500):
        # use planted 1-solution or 2-solution etc? random no-loop too sparse may unsat
```

```

if random.random()<0.5:
    x=[random.choice(labels) for _ in range(n)]
    inst=make_planted_instance_noloops(x)
else:
    base=[random.choice(labels) for _ in range(n)]
    alt=base[:]
    changed=False
    for i in range(n):
        if random.random()<0.3:
            alt[i]=random.choice([y for y in labels if y!=base[i]]); changed=True
    if not changed: alt[0]=random.choice([y for y in labels if y!=base[0]])
    try:
        inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)],
max_attempts=40)
    except RuntimeError:
        continue
    sols=exact_solutions(inst)
    pu,pb=path_consistency(inst)
    if sols and max(len(pu[v]) for v in range(n))>2:
        found=(n,t,len(sols),[len(pu[v]) for v in range(n)],sols[:10],inst)
        print("found",found[:4]); raise SystemExit
print("none")

```

Output:

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[38], line 19
    17 if not changed: alt[0]=random.choice([y for y in labels if y!=base[0]])
    18 try:
--> 19     inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)], max_attempts=40)
    20 except RuntimeError:
    21     continue

Cell In[27], line 20, in make_planted_instance_from_solutions(sol_list, max_attempts)
    18 for v in nonzero:
    19     if v==u: continue
--> 20     for j,sv in enumerate(stubs[v]):
    21         ok=True
    22         for sol in sol_list:

KeyboardInterrupt:

```

Considering path consistency and satisfiability

I'm thinking about testing smaller cases first to see if I can salvage a proof. The idea is to look at the relation graph after applying path consistency (PC). If a variable's unary domain after PC is too large, then extra values should imply possible contradictions. I might explore using a perfect matching approach to prove extendability.

Exploring ternary operation construction

I'm thinking about constructing a fixed ternary operation that preserves relations in loopless networks under PC closure. I'll check if a majority operation on D (based on a repeated majority or smallest label) preserves essential pairwise relations. The goal is to ensure this operation works on typical 2-point and parity relations.

Testing unanimity polymorphism with SAT

I'm considering how unanimity polymorphism works with path consistency. It should extend partial solutions if the constraint relations are invariant under the polymorphism. However, since initial relations might not be, it's unclear. Maybe I can use brute force to check for a majority operation that preserves all raw relations on loopless-support through SAT testing. I'll test with domain size 5 and check for 256 relations using about 4 million constraints.

Testing majority polymorphism with CP-SAT

Code:

```
# Search for ternary majority polymorphism on all raw relations using CP-SAT
from ortools.sat.python import cp_model
model = cp_model.CpModel()
# variable m[x,y,z] in labels
vars_m={}
for x in labels:
    for y in labels:
        for z in labels:
            if x==y or x==z or y==z:
                # majority fixed
                val = x if x==y or x==z else y
                vars_m[(x,y,z)] = val
            else:
                vars_m[(x,y,z)] = model.NewIntVar(1,5,f"m_{x}_{y}_{z}")
# maybe enforce symmetry? not required.
# preserve each raw relation
raw_pairs = [relation_cache[(S,T)] for S in set(port_types) for T in set(port_types)]
# dedupe
raw_pairs_unique=[]
seen=set()
for rel in raw_pairs:
    fr=frozenset(rel)
    if fr not in seen:
        seen.add(fr); raw_pairs_unique.append(rel)
len(raw_pairs_unique)
```

Output:

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
Cell In[39], line 2  
      1 # Search for ternary majority polymorphism on all raw relations using CP-SAT  
----> 2 from ortools.sat.python import cp_model  
      3 model = cp_model.CpModel()  
      4 # variable m[x,y,z] in labels  
  
ModuleNotFoundError: No module named 'ortools'
```

Considering brute-force or z3 for optimization

It seems like using ortools won't work, so I'm thinking about brute-forcing with backtracking, though 60 variables with a domain of 5 might make it difficult. I could also try using z3, but I need to check if it's installed first. Let me verify that.

Checking for z3 installation

Code:

```
import importlib.util, sys  
print(importlib.util.find_spec("z3"))
```

Output:

None

Exploring majority polymorphism and Boolean relations

I'm considering using Python and pulp, but it might not be the best fit. Another idea is that since the relations are defined by Boolean partitions, any majority polymorphism could lead to majority-like behavior. I now wonder if it's enough to satisfy a weaker condition rather than exact membership-majority for all subsets. I'll reason through it.

Refining majority operation and preservation conditions

I'm now thinking that preserving the relations might not need deterministic behavior per partition. Maybe different triples could still lead to valid outputs, depending on other partition bits. I'm wondering if I can force membership-majority by selecting triples of pairs and leveraging

Exploring satisfiability in loopless instances

I'm thinking of focusing on a clearer approach: try proving that if the loopless instance is path-consistent and all unary sizes are ≤ 2 , then it's satisfiable with a count of 2^k . This could be solved using block-flip and parity graphs, but I'm wondering if path consistency allows for larger binary relations while keeping unaries at ≤ 2 .

Examining path consistency and binary relations

I'm considering if size 3 relations might arise in path-consistent loopless structures. With unary size ≤ 2 , binary relations over domains of size ≤ 2 are limited to 1, 2, or 4 pairs. A size 3 relation could appear in a binary domain but seems to break minority closure. Could path consistency eliminate this scenario?

Enumerating binary relation sizes in loopless PC closure

Code:

```
# enumerate restrictions of raw relations to 2x2 domains
two_domains=[set(c) for c in itertools.combinations(labels,2)]
sizes=collections.Counter()
examples={}
for A in two_domains:
    for B in two_domains:
        for rel in raw_pairs_unique if 'raw_pairs_unique' in globals() else [relation_cache[(S,T)]
for S in set(port_types) for T in set(port_types)]:
            res={(x,y) for x,y in rel if x in A and y in B}
            sizes[len(res)] += 1
            examples.setdefault(len(res),(A,B,res))
sizes, {k:examples[k] for k in sorted(examples)[:5]}
```

Output:

```
(Counter({2: 19200, 0: 3200, 4: 3200}),
 {0: ({1, 2}, {1, 2}, set()),
  2: ({1, 2}, {1, 2}, {(1, 1), (2, 1)}),
  4: ({1, 2}, {1, 2}, {(1, 1), (1, 2), (2, 1), (2, 2)}))
```

Classifying size 2 relations on 2x2 domains

I'm looking into the restriction to 2x2 domains, noticing that raw relations can only be of size 0, 2, or 4—never 1 or 3. A size 2 example was a column, not parity. These could be empty, full, unary pinning, or row/column/parity. I might explore how composition/intersection of these relates to 2-SAT.

Classifying size-2 relations on 2x2 domains

Code:

```
# classify size2 relations on 2x2 domains by pattern matrix
patterns=set()
pattern_examples={}
for A in two_domains:
    A1=sorted(A)
    for B in two_domains:
        B1=sorted(B)
        for rel in [relation_cache[(S,T)] for S in set(port_types) for T in set(port_types)]:
            res={(x,y) for x,y in rel if x in A and y in B}
```



```

        if len(res)==2:
            pat=tuple(tuple(1 if (A1[i],B1[j]) in res else 0 for j in range(2)) for i in
range(2))
            patterns.add(pat)
            pattern_examples.setdefault(pat,(A,B,res))
patterns, len(patterns)

```

Output:

```

(((0, 0), (1, 1)),
 ((0, 1), (0, 1)),
 ((0, 1), (1, 0)),
 ((1, 0), (0, 1)),
 ((1, 0), (1, 0)),
 ((1, 1), (0, 0))),
6)

```

Simplifying constraints and using path consistency

I'm thinking that once unary domains are reduced to size ≤ 2 , all constraints may reduce to affine or rectangular relations. Path consistency on this reduced network could simplify things. The key step is ensuring unary domains are ≤ 2 after path consistency, making the problem more manageable and potentially solvable in polynomial time.

Pinning variables and simplifying the problem

If a variable's domain size is ≤ 2 , trying all 5 labels and using path consistency might help isolate components. This could potentially solve the unsatisfiable (unsat) recovery issue. After pinning one variable, perhaps the solution set in the remaining blocks becomes easier to handle, maybe even a linear subspace.

Evaluating solution count strategies

Projections could target exact solutions, but it's tricky. Pinning variables could help, yet I'm not sure if it'll determine enough. Pinning root variables for each label and counting might yield the number of solutions, but recursion depth could be problematic. Perhaps a more efficient approach is needed, like using inclusion-exclusion or linear algebra after each pin.

Considering a verification-based algorithm

I'm thinking about solving satisfiability by testing 5 pinned instances per variable, which reduces to a polynomial number of tests. If PC works on loopless cases, it might simplify the process. The full algorithm would use PC to pin variables, then reconstruct relations and count solutions. We might be able to test it, but I'm unsure about how loops would affect accuracy.

Testing loopless cases for completeness

I'm considering whether width 1 is strictly complete, especially after finding a SAC counterexample with loops. It might hold true for loopless cases, though. I think we should test small, loopless instances by checking satisfiability for each value. This could help identify any counterexamples. Let's go ahead and search for them.

Searching for strict width and SAC counterexamples

Code:

```
def pin_instance(inst, v, val):
    n=inst["n"]
    unary={u:set(inst["unary"][u]) for u in range(n)}
    unary[v]&={val}
    return {"n":n, "unary":unary, "binary":{k:set(vv) for k,vv in inst["binary"].items()},
"edges":inst["edges"], "loops":inst["loops"]}

def pc_nonempty(inst):
    pu,pb=path_consistency(inst)
    return all(pu[v] for v in range(inst["n"]))

# search no-loop SAC counterexample
random.seed(12)
found=False
for n in range(2,7):
    # use random no-loop and planted multisolition if possible
    for t in range(200):
        if random.random()<0.7:
            # random or planted unique
            if random.random()<0.5:
                x=[random.choice(labels) for _ in range(n)]
                inst=make_planted_instance_noloops(x)
            else:
                inst=make_random_instance_noloops(n)
        else:
            base=[random.choice(labels) for _ in range(n)]
            alt=base[:]
            for i in range(n):
                if random.random()<0.4:
                    alt[i]=random.choice([y for y in labels if y!=base[i]])
            try:
                inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)],
max_attempts=20)
            except:
                continue
        sols=exact_solutions(inst)
        # maybe skip very large solution count? small n okay
        for v in range(n):
            for val in labels:
                sat = any(sol[v]==val for sol in sols)
                pne = pc_nonempty(pin_instance(inst,v,val))
                if sat != pne:
```

```
print("SAC counterexample no-loop
n",n,"trial",t,"v",v,"val",val,"sat",sat,"pc",pne,"#sol",len(sols))
found=True
raise SystemExit
print("n",n,"done")
print("none found")
```

Output:

```
n 2 done
n 3 done
n 4 done
n 5 done
n 6 done
none found
```

Exploring potential algorithms and checking exactness

It seems there's strong evidence that SAC could be exact on loopless EO instances, not PC. I need to conduct further systematic tests, especially with $n=6$. The exact solutions grow quickly, but focusing on pinned satisfiability should help. Proving singleton-PC exactness for loopless cases might be simpler—let's consider it combinatorially.

Considering imbalance argument for pinned instances

I'm thinking of extending the earlier imbalance argument to pinned instances. If we pin a value, AC/PC should give a contradiction if the pin is wrong. In a loopless AC fixed point, a singleton variable implies others have domains of size 1 or 2, and the instance is satisfiable by greedy construction.

Exploring geometry and path consistency for pinned instances

I'm thinking about defining a signature for domain sets and how path consistency or pinning interacts with this. If we pin a singleton, the affine solution space reduces, and the geometry of projections might help decide satisfiability. It could be enough to solve the linear skeleton and check intersection with the graph.

Exploring block computation using homogeneous systems

I'm thinking about solving for blocks independently of the base solution, especially for pinned instances. By fixing the root variable and analyzing the homogeneous difference system, we might be able to compute block relations and satisfiability. Using linear equations and secant directions could help decide the solution directly.

Simplifying the pinned instance solution approach

For the pinned instance, the feasible value set stays small, limiting domain sizes. Maybe the solution set is a *subdirect product* of small domains, and arc or path consistency can solve it. If we run an incremental algorithm, we could iteratively test and solve the Boolean network by branching on domain values when necessary.

Exploring pinning and consistency bounds

If path consistency after a root pin still leaves domains larger than 2, maybe there are only a few such variables—possibly none in loopless instances. I wanted to check how the global imbalance equation could help bound the number of large domains, particularly in 2-solution, no-loop instances.

Exploring pairwise compatibility through path consistency

The set $P_{A,B}$ defines bit pairs depending on the intersection of A and B . Composing along a path of length 2 links to potential constraints on port bit pairs. Path consistency may capture the compatibility between pairs of bits across labels, potentially reducing options to ≤ 2 large enough to distinguish labels.

Exploring propagation of constraints through neighbors

Path consistency via a neighbor could propagate constraints on bit pairs. In a loopless graph with a root pin, we might be able to test every pair of port types through a 2-step path, recovering the row relation and label set. If ports p and q connect to the same neighbor, path consistency could limit possible values for x_v .

Building an algorithm with graph decomposition

I'm wondering if I could build an algorithm using the connectedness of a "port-type network" rather than relying on path consistency. Every vertex has 56 ports, and edges lead to equalities of homogeneous bits. Each root label defines compatible labels across paths, helping to calculate decisions via SAC.

Exploring root-pinned path consistency and SAC

If the root is pinned, path consistency (PC) should give unary domains at each vertex that are compatible with the root label along every path. I wonder if SAC for loopless instances could be equivalent to consistency across pair relations, possibly using graph homomorphism. Could the solution be constructed from allowed labels, ensuring consistency across blocks? I'll need to check on that.

Considering root pin and block structure for consistency

If each label is an affine function of block bits relative to the root, pair relations might depend on whether the root-to- v and root-to- u secant directions are in the same block. But root pin doesn't confirm block structure on

Considering the computational process with root pinning

I'm considering how to compute for blocks that don't touch the root. I could explore one representative variable from each component. Perhaps I could reconstruct the parity graph from pinned PC, even if unpinned PC isn't exact. If singleton-PC is exact, the algorithm seems polynomial and efficient. I'll work on proving that in a satisfiable pinned instance, PC closure equals true projections.

Exploring solution relations and path consistency

Since the root is fixed, I'm considering if the solution relation on remaining variables still behaves like a block flip, where each block is either unaffected (2-valued) or fixed by the root (1-valued). For any value left in the PC domain of a variable, I aim to show that there's a solution. Path consistency computes constraints along all root-to-variable paths, and with the root's domain being singleton, the unary domain at v simplifies. However, Helly's theorem might not fully apply here.

Exploring the interaction of PC closure and 2-decomposability

After fixing the root, I wonder if the remaining solutions preserve the relation through a majority operation, which is conservative on the unary domains. Since unary domains are size ≤ 2 , this creates a near-unanimity polymorphism, making the relation 2-decomposable. The goal is to see if PC closure, which computes the minimal 2-decomposable superset of solutions, matches the true 2-decomposable solution relation. Let's formalize this.

Clarifying path consistency closure and solution relation

$\text{Sol}(P)$ includes global assignments consistent with all binary closed constraints, but it may or may not be non-empty. Path consistency removes incompatible pairs, preserving all valid solutions. If the true solution relation is 2-decomposable and fits within the closure, then it's possible $\text{Sol}(P)$ could be larger. I need to show that every tuple in $\text{Sol}(P)$ is indeed a solution, considering the majority preservation and possible use of "microstructure graph" or Helly's property.

Exploring feasible counterexamples for singleton-PC exactness

It seems that assignments would extend on majority-closed domains, but that's still not enough to confirm things. I think searching for a counterexample to singleton-PC exactness for larger n using a heuristic SAT/backtracking approach might help — not exhaustive, but manageable up to $n=7$, given 78125 assignments per instance.

Searching no-loop SAC counterexamples with SAT

Code:

```
# more search no-loop SAC counterexample up to n=7 with unique/planted instances
random.seed(13)
found=False
for n in range(2,8):
    trials=100 if n<=6 else 30
    for t in range(trials):
```

```

# mix random and planted
mode=random.random()
if mode<0.4:
    inst=make_random_instance_noloops(n)
elif mode<0.7:
    x=[random.choice(labels) for _ in range(n)]
    inst=make_planted_instance_noloops(x)
else:
    base=[random.choice(labels) for _ in range(n)]
    alt=base[:]
    changed=False
    for i in range(n):
        if random.random()<0.4:
            alt[i]=random.choice([y for y in labels if y!=base[i]]); changed=True
    if not changed: alt[0]=random.choice([y for y in labels if y!=base[0]])
    try:
        inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)],
max_attempts=10)
    except:
        continue
sols=exact_solutions(inst) if n<=7 else None
if sols is None:
    continue
for v in range(n):
    for val in labels:
        sat = any(sol[v]==val for sol in sols)
        pne = pc_nonempty(pin_instance(inst,v,val))
        if sat != pne:
            print("counterexample
n",n,"trial",t,"v",v,"val",val,"sat",sat,"pc",pne,"#sol",len(sols))
            found=True
            raise SystemExit
print("n",n,"done")
print("none found")

```

Output:

```

n 2 done
n 3 done
n 4 done
n 5 done
n 6 done
n 7 done
none found

```

Testing search size after singleton-PC

I think this could work as evidence, but I need to test if the search size grows after singleton-PC, especially with domains greater than 2. Maybe pinned PC results in unary domains of size ≤ 2 . I'll test this theory to see if it holds up.

Testing domain size and unary constraints

Code:

```
random.seed(14)
max_domain=0
example=None
for n in range(2,8):
    for t in range(80):
        mode=random.random()
        if mode<0.4:
            inst=make_random_instance_noloops(n)
        elif mode<0.7:
            x=[random.choice(labels) for _ in range(n)]
            inst=make_planted_instance_noloops(x)
        else:
            base=[random.choice(labels) for _ in range(n)]
            alt=base[:]
            for i in range(n):
                if random.random()<0.4:
                    alt[i]=random.choice([y for y in labels if y!=base[i]])
            try:
                inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)],
max_attempts=10)
            except:
                continue
    sols=
```

Output:

```
max_domain 2 example (4, 10, 0, 1, 2, 2)
```

Testing pinned PC binary relations on 2x2 domains

It's interesting—so far, in all no-loop pinned satisfiable instances, PC reduced all domains to size ≤ 2 . This is a key finding. Now, I want to check if pinned PC binary relations on 2x2 domains always have sizes of 0, 1, 2, or 4, and not 3. It seems likely, but I'll test it.

Testing pinned PC binary relations over 2x2 domains

Code:

```
random.seed(15)
bad=False
for n in range(2,8):
    for t in range(60):
        mode=random.random()
        if mode<0.4:
            inst=make_random_instance_noloops(n)
```

```

elif mode<0.7:
    x=[random.choice(labels) for _ in range(n)]
    inst=make_planted_instance_noloops(x)
else:
    base=[random.choice(labels) for _ in range(n)]
    alt=base[:]
    for i in range(n):
        if random.random()<0.4:
            alt[i]=random.choice([y for y in labels if y!=base[i]])
    try:
        inst=make_planted_instance_from_solutions([tuple(base),tuple(alt)],
max_attempts=10)
    except

```

Output:

```

none bad

```

Clarifying my algorithm and loops

OK, so it seems I can present an algorithm more concretely for loopless instances, but for general cases (with loops), it's more conjectural. I should be clear that the current results apply to loopless settings. I can't claim loops are handled without confirmation, though—loops seem to introduce issues that break path consistency, based on the analysis.

Exploring role of loops in hardness

It seems that unary gadgets, which are typically found in EO problems with loops, could be the key to the hardness aspect. In loopless problems, these would be disallowed. Maybe the best approach is to first contract loops before solving the general problem. Yet, loops might still complicate the complexity hierarchy. We've seen simple consistency struggles with loops, so this is still unfinished.

Considering loop handling and consistency

I'm thinking of preprocessing loops by working with unary domains first, as they could simplify the setup. After pinning, perhaps the algorithm can handle them with SAC/PC. The loop itself might cause more complexity, but I suspect splitting vertices could potentially reduce looped instances to simpler, loopless problems. More testing is definitely needed.

Evaluating the role of loops and unary gadgets

I'm considering using a dummy vertex to simulate the unary effect of loops, but there's a question of whether this changes the complexity. If loops require special constraints, I think the conjecture could break. However, using a loopless approach could potentially expose contradictions in earlier assumptions. More analysis is needed.

Summarizing loopless EO results and candidate algorithm

I've discovered that for loopless EO instances, singleton path consistency on a pinned value is exact in most cases, reducing every domain to size ≤ 2 . This provides a concrete algorithm involving 5 root pins and counting feasible solutions. However, loops still create complications, with counterexamples where AC/SAC/PC fail. The next step is proving singleton-PC exactness.

Reading documents

Reading documents

Addressing citation concerns with filecite markers

I'm wondering if I should use filecite markers from previous snippets instead of file_search citations. However, I should follow the developer's instructions for citations and provide line-range citations when using file_search results. I'll try using mclick to gather line ranges before proceeding.

Reading documents

Formulating a clear response and progress update

I need to provide a final answer, being upfront and honest about the current state of research. I'll use filecite markers for citations and mention that my computational experiments do not require external citations. I'll update the user on progress, acknowledge the gap, and structure the response with the next steps and a tentative conclusion.

Explaining the mathematical details of the binary relations

The algorithm is supported for loopless EO, with counterexamples found for general loops regarding PC/SAC. I should include why binary relations on 2-point fibers are affine: the restriction on any raw relation has size 0, 2, or 4, leading to patterns. I'll prove that by analyzing the forms of bit relations.